

CS 4240: Compilers and Interpreters, FUN Project, Fall 2025

Update (10/28): Deadline: Please submit it by 11/24.

Update (9/23): IMPORTANT: Your `yours.c` should not contain `loop/goto` statements.

Update (9/23): You do not need to use LLVM-19.1.0. Other versions should be fine because I will not grade your submission.

Update (9/23): This project won't be graded, and it's just for fun with LLMs. Please don't spend time doing it manually!

1 Project Description

This project simulates a practical code optimization scenario. The goal is to optimize the function in the provided C file (`yours.c`), ensuring it is functionally equivalent to the reference implementation. Performance will be assessed using the `llvm-mca` tool on the AMD Zen2 architecture. In principle, you can complete the project on a non-X86 machine. However, we recommend using Linux on an X86 machine. The project uses LLVM-19.1.0 [1], which includes both the `clang` compiler and the `llvm-mca` performance evaluation tool.

In the project folder, a reference implementation is available in the file `impl.c`. We also provide a testing file `tester.c` to check the equivalence of `impl.c` and `yours.c` using 10 random inputs. We will evaluate the performance of your optimized C file `yours.c` using the reciprocal throughput (Block RThroughput) reported by `llvm-mca`.

To sum up, your optimized implementation in `yours.c` will be assessed by `clang` and `llvm-mca` in LLVM-19.1.0 according to the following criteria:

- Optimized function in `yours.c` should be equivalent to the reference function in `impl.c`;
- The function in `yours.c` terminates and is free of undefined behaviors [2];
- The reciprocal throughput (reported by `llvm-mca`) of the optimized function in `yours.c` should be reasonably small.

1.1 Evaluation

We will take the following steps to evaluate your project:

1. We will compile your submitted file using `clang-19.1.0`;
2. We will run the executable produced by `tester.c` to test the equivalence of `yours.c` and `impl.c`.
3. We will check if `yours.c` is free of undefined behaviors;
4. We will run `llvm-mca` on your submitted file `yours.c` with the corresponding `znver2` flags to assess its performance on the AMD Zen2 architecture.

The provided script `run.sh` will perform the aforementioned evaluation steps. We will use this script to evaluate your submission.

- You will receive 50% of the credit if the reported RThroughput is smaller than 110; or
- You will receive 100% of the credit if the reported RThroughput is smaller than 90.

2 Provided Code and Expected Output

We have provided the project files at <https://faculty.cc.gatech.edu/~qrzhang/course/cs4240/bonus25.tar.bz2>. The project folder contains the following files:

- `impl.c`: a reference implementation.
- `yours.c`: the main file to work on. The original file is almost identical to `impl.c`.
- `tester.c`: it includes both `impl.c` and `yours.c` to test their outputs.
- `run.sh`: the main script for project evaluation.

You need to update the paths of `CC` and `MCA` in `run.sh`. The default output is given as follows:

```
$ bash run.sh
243.8
```

Note that you are only allowed to perform source code optimization directly in `yours.c` or modify the compiler optimization flags `CFLAGS` in `run.sh`. You are not allowed to change the testing environment (such as `run.sh`, `tester.c` or `impl.c`) or link with external libraries.

2.1 Useful Suggestions (but may not be useful for Fall 2025)

- A prerequisite is to build the correct LLVM version. We have provided a link to the specific source code of LLVM [1].
- You can change `CFLAGS` in the provided `run.sh` script. We will only use the provided `run.sh` with your `CFLAGS` to evaluate your submission.
- To check the correctness, you can compile and run `tester.c` on your local machine. However, to assess the performance, you have to use the corresponding `znver2` flags given in `run.sh`.
- Do not use external libraries.

- Passing `run.sh` does not necessarily mean that your `yours.c` is free of undefined behaviors.

3 Submission

Send an email to the instructor with the title “[CS 4240 FUN Project] YOUR NAME(s)” that contains:

- The optimized `yours.c` file;
- The optimization flag `CFLAGS` used in `run.sh`; and
- A document that describes the key steps to obtain the final `yours.c` file.

~~If your submission has correctness issues (e.g., it contains undefined behavior, or it produces incorrect outputs), we will inform you. In this case, you can submit it again. However, your second submission will be final. Note that there is no need to submit your `run.sh` because we will use the default one to evaluate your submission.~~

4 Collaboration

You are free to work with anyone, including non-CS4240 students.

~~For this BONUS project, you can form a team of *at most* two people.~~

~~We will award identical grades to each member of a given project team, unless members of the team directly register a formal complaint. We assume that the work submitted by each team is their work solely. Any clarification questions about the project handout should be posted on the course’s public Piazza message board. Under no condition is it acceptable to use code or source written by another team.~~

References

- [1] LLVM 19.1.0. <https://github.com/llvm/llvm-project/releases/tag/llvmorg-19.1.0>.
- [2] Undefined behavior. https://en.wikipedia.org/wiki/Undefined_behavior.